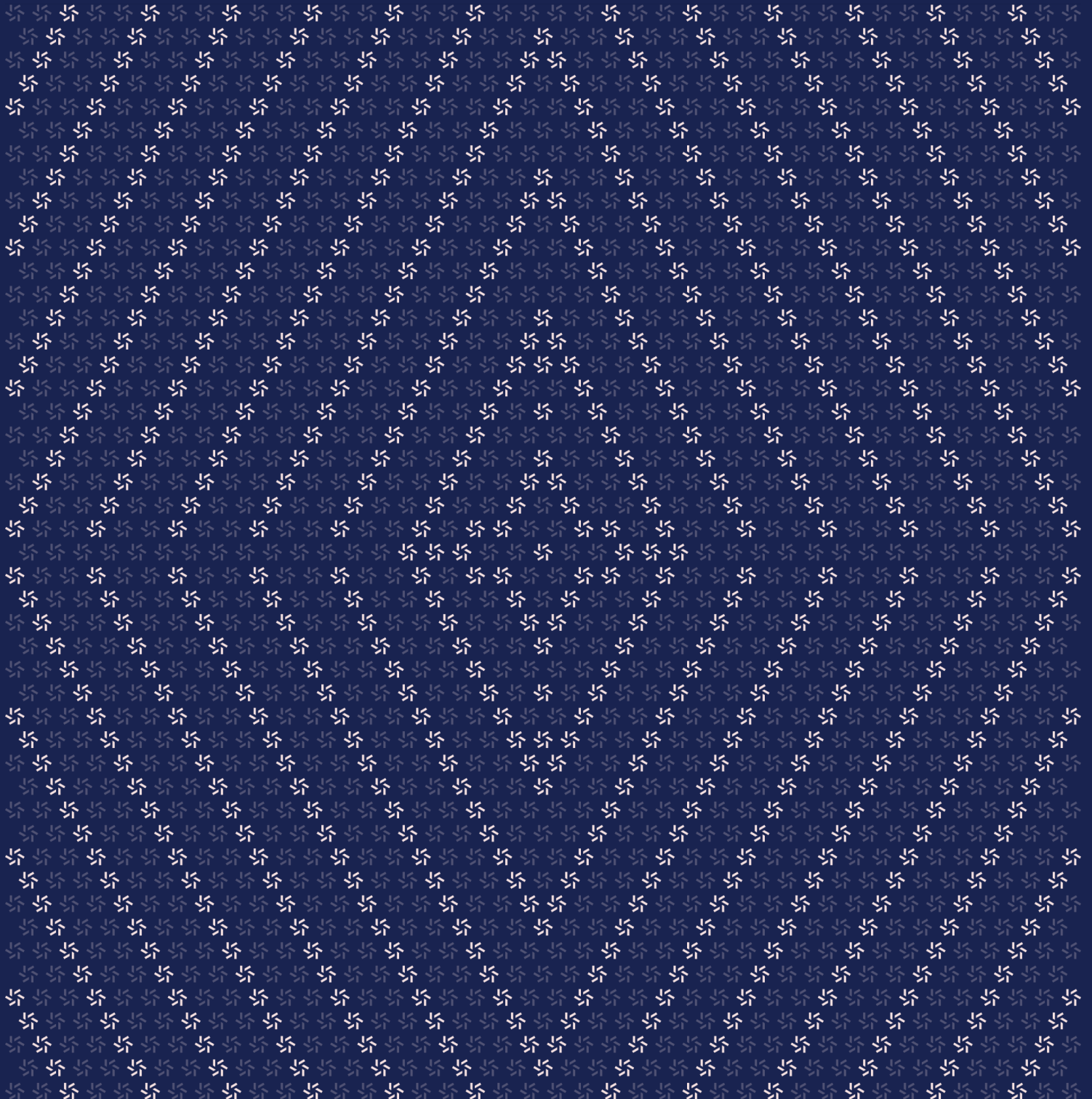


September 12, 2024

StablecoinTemplateV2

Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	4
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	6
2.1. About StablecoinTemplateV2	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	10
3.1. Missing administrator address check for zero address	11
3.2. Griefing attack on minting	13
4. Discussion	14
4.1. Centralization risk	15
4.2. Lack of test case on contract upgradability	15

5.	Threat Model	15
5.1.	Module: StablecoinTemplateV2.sol	16
6.	Assessment Results	21
6.1.	Disclaimer	22

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Bridge Building Inc. from September 9th to September 10th, 2024. During this engagement, Zellic reviewed StablecoinTemplateV2's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Could non-administrators execute administrators' functions?
 - Could more tokens be minted than the maximum supply?
 - Is access control done properly for each role?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

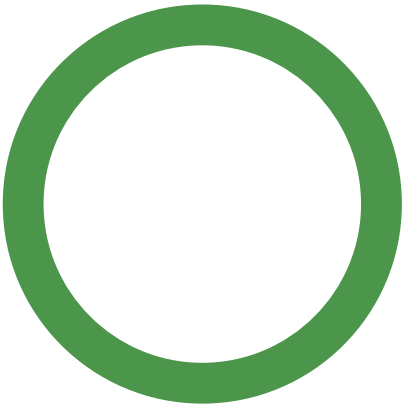
1.4. Results

During our assessment on the scoped StablecoinTemplateV2 contracts, we discovered two findings, both of which were low impact.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Bridge Building Inc. in the Discussion section ([4. 7](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	2
Informational	0



2. Introduction

2.1. About StablecoinTemplateV2

Bridge Building Inc. contributed the following description of StablecoinTemplateV2:

StablecoinTemplateV2 serves as a template project for releasing fully reserved proprietary stablecoins pegged to the US dollar, all managed by Bridge Building Inc.

Bridge Building Inc. intends to launch multiple stablecoins using this codebase. The only variations among these projects will be their respective token names, symbols, and decimals.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case

basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4. 7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

StablecoinTemplateV2 Contracts

Type	Solidity
Platform	EVM-compatible
Target	stablecoin-template
Repository	https://github.com/withbridge/stablecoin-template
Version	c536871badba04acad7c6782f1f2a650712a27ca
Programs	StablecoinTemplateV2.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of one person-day. The assessment was conducted by two consultants over the course of two calendar days.

Contact Information

The following project managers were associated with the engagement:

Jacob Goreski
✈ Engagement Manager
jacob@zellic.io ↗

Chad McDonald
✈ Engagement Manager
chad@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Juchang Lee
✈ Engineer
lee@zellic.io ↗

Sylvain Pelissier
✈ Engineer
sylvain@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

September 9, 2024	Kick-off call
--------------------------	---------------

September 9, 2024	Start of primary review period
--------------------------	--------------------------------

September 10, 2024	End of primary review period
---------------------------	------------------------------

3. Detailed Findings

3.1. Missing administrator address check for zero address

Target	StablecoinTemplateV2		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The initialize function is called once during contract creation and cannot be called later. This function gives the administrator role to an address passed as a parameter:

```
function initialize(string calldata name, string calldata symbol,
    uint8 decimals, address admin) initializer public {
    _maxSupply = 0;
    _adminRoleCount = 0;
    _decimals = decimals;

    __ERC20_init(name, symbol);
    __Pausable_init();
    __Ownable_init();
    __AccessControl_init();
    __ERC20Permit_init(name);
    __UUPSUpgradeable_init();

    _grantRole(DEFAULT_ADMIN_ROLE, admin);
}
```

The administrator has a lot of privileges and has a central role for the contract, but during initialization, the admin address is not checked to be nonzero in the _grantRole function:

```
/**
 * @dev Grants `role` to `account`.
 */
function _grantRole(bytes32 role, address account)
internal virtual override {
    bool accountAlreadyHasRole = hasRole(role, account);

    super._grantRole(role, account);

    if (role == DEFAULT_ADMIN_ROLE && !accountAlreadyHasRole) {
        _adminRoleCount += 1;
    }
}
```

```
    }  
}
```

Another way of promoting an address to the administrator role is by calling, from another administrator address, the `grantRole` function inherited from the `AccessControlUpgradeable` contract. In this case, the `_grantRole` will be called again without any check.

Impact

If by accident, the `admin` address is passed as zero in the `initialize` or `grantRole` functions, then the contract may be left without administrators. The `_revokeRole` function could revoke all other administrators' roles, and only the zero address may remain. Bridge Building Inc. notes that a developer could always deploy a new contract with a proper admin address if this occurs.

Recommendations

To avoid accidentally having no admin for the contract, a zero-address check should be implemented in the `initialize` and `grantRole` functions.

Remediation

This issue has been acknowledged by Bridge Building Inc., and a fix was implemented in commit [62ffe390](#).

3.2. Griefing attack on minting

Target	StablecoinTemplateV2		
Category	Coding Mistakes	Severity	Low
Likelihood	Medium	Impact	Low

Description

To mint tokens, an address with the `MINTER_ROLE` role calls the `mint` function. This function calls the `_mint` function of the `ERC20Upgradeable` contract:

```
function _mint(address account, uint256 amount) internal virtual {
    require(account != address(0), "ERC20: mint to the zero address");

    _beforeTokenTransfer(address(0), account, amount);

    _totalSupply += amount;
    unchecked {
        // Overflow not possible: balance + amount is at most totalSupply +
        // amount, which is checked above.
        _balances[account] += amount;
    }
    emit Transfer(address(0), account, amount);

    _afterTokenTransfer(address(0), account, amount);
}
```

The function `_beforeTokenTransfer` is overridden in the `StablecoinTemplateV2` contract to check that the sender and receiver addresses are not blocked and the contract is not paused:

```
function _beforeTokenTransfer(address from, address to, uint256 amount)
    internal
    whenNotPaused
    whenNotInBlockedList(from)
    whenNotInBlockedList(to)
    override
    {
        super._beforeTokenTransfer(from, to, amount);
    }
```

An address with the BLOCKER_ROLE can add the zero address in the blocked list. Then, the function `_mint` would revert on the call to `_beforeTokenTransfer(address(0), account, amount)` since the zero address is in the blocked list. The mint operation would never happen. The same issue applies to the burn function.

Impact

Even if address roles have separated privileges, an address with the BLOCKER_ROLE can prevent minting by adding the zero address in the blocked list. Nevertheless, the impact is low since the administrator of the contract can revoke the malicious blocker address or another blocker address can remove the zero address from the blocker list, allowing to mint again.

Recommendations

Depending on the use cases, for minting or burning operations, the zero address may not be checked to be in the blocked list or it should not be possible to add the zero address to the blocked list.

Remediation

This issue has been acknowledged by Bridge Building Inc., and a fix was implemented in commit [c6557b51](#).

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Centralization risk

There are eight distinct types of privileged accounts for the StablecoinTemplateV2 contract:

- The default administrators
- The minters
- The pausers
- The unpausers
- The blockers
- The unblockers
- The blocked address burners
- The owner

All the roles have self-explaining names, but it should be noted that the default administrators have the power to revoke or grant roles to any addresses, even for their own administrator role.

The above introduces centralization risks that users should be aware of, as it grants a single point of control over the system. We recommend that these centralization risks be clearly documented for users so that they are aware of the extent of the owner's control over the contract. This can help users make informed decisions about their participation in the project. Additionally, clear communication about the circumstances in which the owner may exercise these powers can help build trust and transparency with users.

Note that Bridge Building Inc. took additional measures to mitigate these risks; a multi-signature is used for administrator and all other roles' access.

4.2. Lack of test case on contract upgradability

There was no test case covering the upgradability of the StablecoinTemplateV2 contract. The test has been written and verified during the audit. Bridge Building Inc. has confirmed this and will add it to the test suite.

This issue has been acknowledged by Bridge Building Inc., and a fix was implemented in commit [2e4474a0](#).

5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

5.1. Module: StablecoinTemplateV2.sol

Function: burn()

Addresses having the MINTER_ROLE can use the burn function to burn tokens.

Inputs

- amount
 - **Validation:** Must be less than the account's balance.
 - **Impact:** The amount of tokens to be burnt.

Branches and code coverage (including function calls)

Intended branches

- Address with a minter role burns less tokens than its balance.
 - ☒ Test coverage

Negative behavior

- Burning reverts when the sender does not have the minter role.
 - ☒ Negative test
- Burning reverts more than the maximum supply of token is minted.
 - ☒ Negative test
- Burning reverts when the sender is blocked.
 - ☒ Negative test

Function call analysis

- `_burn(msgSender(), amount)`
 - **External/Internal?** Internal.
 - **Argument control?** amount is controlled.
 - **Impact:** Call to the ERC20Upgradeable burning function.

Function: `burnFromBlockedAddress()`

Addresses having the `BLOCKED_ADDRESS_BURNER_ROLE` can use the `burnFromBlockedAddress` function to burn all the tokens owned by a blocked address.

Inputs

- `account`
 - **Validation:** The account is checked to be in the blocked list. The balance of the account is checked to be nonzero.
 - **Impact:** The account to be emptied.

Branches and code coverage (including function calls)

Intended branches

- Tokens of a blocked address are burnt.
☒ Test coverage

Negative behavior

- Burning reverts when the address is not blocked.
☒ Negative test
- Burning reverts when the sender does not have the `BLOCKED_ADDRESS_BURNER_ROLE` role.
☒ Negative test
- Burning reverts when the sender balance is zero.
☒ Negative test

Function call analysis

- `_burn(account, accountBalance)`
 - **External/Internal?** Internal.
 - **Argument control?** `account` is controlled.
 - **Impact:** Call to the `ERC20Upgradeable` burning function.

Function: `initialize()`

The goal of the `initialize` function is similar to a constructor for a base contract deployed behind a proxy. It can be called only once due to the `initializer` modifier.

Inputs

- `name`

- **Validation:** No validation.
 - **Impact:** The name of the token.
- symbol
 - **Validation:** No validation.
 - **Impact:** The symbol of the token.
- decimals
 - **Validation:** No validation.
 - **Impact:** The decimals of the token.
- admin
 - **Validation:** No validation.
 - **Impact:** The address of the first default administrator.

Branches and code coverage (including function calls)

Intended branches

- A contract is initialized with a valid name, symbol, and administrator address.
 - ☒ Test coverage

Negative behavior

- A contract is initialized with the zero address as the administrator.
 - ☐ Negative test

Bridge Building Inc. has acknowledged the issue and added testcases in [62ffe390](#) ↗

Function call analysis

- `_grantRole(DEFAULT_ADMIN_ROLE, admin)`
 - **External/Internal?** Internal.
 - **Argument control?** `admin` is controlled.
 - **Impact:** Grants the `admin` address the administrator role.

Function: `mint()`

Addresses having the `MINTER_ROLE` can use the `mint` function to mint tokens. Some checks are applied before calling the inherited `_mint` function.

Inputs

- `to`
 - **Validation:** The address is checked to be on the minter-recipient list.

- **Impact:** The address to send the minted tokens.
- amount
 - **Validation:** The amount of tokens minted together with the total supply amount is checked to be less than the maximum supply.
 - **Impact:** The amount of tokens to be minted.

Branches and code coverage (including function calls)

Intended branches

- Address with the minter role mints to a receiver on the mint-recipient list.
 - ☒ Test coverage

Negative behavior

- Minting reverts when the receiver is not on the mint-recipient list.
 - ☒ Negative test
- Minting reverts when the sender does not have the minter role.
 - ☒ Negative test
- Minting reverts when more than the maximum supply of tokens is minted.
 - ☒ Negative test
- Minting reverts when the receiver is blocked.
 - ☒ Negative test
- Minting reverts when the minter is blocked.
 - ☐ Negative test

Bridge Building Inc. stated the following in relation to the missing negative test: This is not possible because the minted tokens are coming from the 0x0 address. The way we block a minter is by revoking its minter role. Minting without a minter role has a negative test.

Function call analysis

- `_mint(to, amount)`
 - **External/Internal?** Internal.
 - **Argument control?** `to` and `amount` are controlled.
 - **Impact:** Call to the ERC20Upgradeable minting function.

Function: `_grantRole()`

The `_grantRole` function overrides the implementation of `OpenZeppelin` in `AccessControlUpgradeable.sol`. This function is used to grant a role. This function is called by the `grantRole` function inherited from `AccessControlUpgradeable.sol`. If the role to be granted is `DEFAULT_ADMIN_ROLE`, the contract keeps track of the total number of administrators to avoid having no administrator left dur-

ing revocation.

By default, the administrator role for all roles is DEFAULT_ADMIN_ROLE. It is possible to set administrator roles for selected roles with the function `_setRoleAdmin`, but this is not used by the contract. Thus, only the administrators are able to grant or revoke roles.

Inputs

- `role`
 - **Validation:** No validation.
 - **Impact:** The role identifier to revoke.
- `account`
 - **Validation:** No validation.
 - **Impact:** The account to revoke.

Branches and code coverage (including function calls)

Intended branches

- Existing roles are granted by the administrator.
 - ☒ Test coverage
- Several administrator roles are granted.
 - ☒ Test coverage

Negative behavior

- A role is not granted when the account is not an administrator.
 - ☒ Negative test
- Nonexisting roles are granted by the administrator.
 - ☐ Negative test

Function call analysis

- `super._grantRole(role, account)`
 - **External/Internal?** Internal.
 - **Argument control?** `role` and `account` are controlled.
 - **Impact:** Call to the base contract granting the function

Function: `_revokeRole()`

The `_revokeRole` function overrides the base contract implementation in `AccessControlUpgradeable.sol`. This function is used to revoke a role and also to check that if the role revoked is `DEFAULT_ADMIN_ROLE`, then at least one account with the administrator role will remain after the revocation. Otherwise, the revocation reverts. This function is called by the `revokeRole` function in-

herited from AccessControlUpgradeable.sol.

Inputs

- `role`
 - **Validation:** No validation.
 - **Impact:** The role identifier to revoke.
- `account`
 - **Validation:** No validation.
 - **Impact:** The account to revoke.

Branches and code coverage (including function calls)

Intended branches

- Accounts having existing roles are revoked by the administrator.
 - ☒ Test coverage

Negative behavior

- Revoke an administrator role and there is only one administrator left.
 - ☒ Negative test
- Nonexisting roles are revoked by the administrator.
 - ☐ Negative test

Function call analysis

- `super._revokeRole(role, account)`
 - **External/Internal?** Internal.
 - **Argument control?** `role` and `account` are controlled.
 - **Impact:** Call to the base contract revoking function.

6. Assessment Results

At the time of our assessment, the reviewed code was not deployed to the Ethereum Mainnet.

During our assessment on the scoped StablecoinTemplateV2 contracts, we discovered two findings, both of which were low impact.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.